MIVA OPEN UNIVERSITY

SCHOOL OF COMPUTING & APPLIED SCIENCES • DEPARTMENT OF CYBERSECURITY

COS 106: INTRODUCTION TO WEB TECHNOLOGIES

PRACTICAL TERM PROJECT ASSESSMENT REPORT (20 MARKS)

Design and Development of a Responsive Student Portfolio and Academic Management Platform



Full Name:	PRINCE DANIEL
Matriculation No:	2025/C/CYB/1333
Student ID:	301897306
Programme & Level:	B.Sc. Cybersecurity • 100 Level
Student Email:	p.daniel7306@miva.edu.ng
Phone Number:	+234 811 676 1295
Campus Location:	Abuja, Federal Capital Territory, Nigeria
Technical Specialization:	Frontend Web Technologies & Mobile App Development
Color Palette:	Royal Navy Blue (#0A192F) • Electric Cyan (#38BDF8)
Date of Submission:	November 2026

Submitted in partial fulfillment of the academic assessment requirements for COS 106: Introduction to Web Technologies (20 Marks).

1. Introduction & Project Overview

As a 100-Level Cybersecurity undergraduate at MIVA Open University, I engineered this practical term project for **COS 106: Introduction to Web Technologies**. The mandate was to build a comprehensive, multi-page website that functions as a personal developer portfolio and as an interactive management platform for student coursework and academic deadlines.

The platform was developed from the ground up using pure **HTML5**, **CSS3**, and **Vanilla JavaScript (ES6+)** without external CSS libraries or frameworks. This zero-dependency architecture highlights fundamental mastery of semantic document outlines, responsive CSS Flexbox and Grid layouts, client-side input validation, dynamic event listeners, and persistent browser storage via `localStorage`.

> **Curriculum & Course Code Clarification:** In the official MIVA Open University curriculum and assessment brief, this course is designated as **COS 106: Introduction to Web Technologies** (20 Marks). On the student LMS dashboard portal, the active course shell is titled **CSC 106 – Introduction to Web Technologies**. Both represent the identical undergraduate curriculum in the School of Computing. Throughout this report and codebase, **COS 106** is maintained as the primary title in strict adherence to the assessment brief.

2. Multi-Page Architecture & Page Breakdown

2.1 Homepage (`index.html`)

The homepage is uniquely architected with an asymmetric hero layout featuring the student profile card and portrait positioned on the **left side**, paired with greeting copy and action buttons on the right. It includes an academic KPI grid highlighting 100-Level standing, 8 enrolled courses, target cumulative GPA (4.80 / 5.00), and technical focus on Web & Mobile engineering.

2.2 About Me Page (`about.html`)

Provides Prince Daniel's academic background and technical profile:

- **Coursework Credit Summary Table:** A semantic HTML table detailing all eight registered 100-Level courses (COS 106 / CSC 106, COS 102, MIVA-COS 111, MTH 102, PHY 102, PHY 108, GST 112, GST 122) totaling 19 credit units with classifications and target grades.
- **3-Phase Professional Roadmap:** Outlining progression from Web/Mobile foundations to secure full-stack development and senior application security architecture.
- **Animated Skill Bars:** Smooth CSS fill animations triggered when scrolled into view using the modern `IntersectionObserver` API.
- **Multimedia Integration:** Native HTML5 `<audio controls>` voice note introduction and a responsive `<video controls>` walkthrough.

2.3 Projects Showcase Page (`projects.html`)

Features four distinct applied computing and cybersecurity systems:

1. **ApexPay Mobile Wallet & Airtime Hub:** Cross-platform Flutter fintech interface supporting utility settlements and biometric transaction logs.
2. **CampusConnect Event & Ticket Hub:** Responsive event booking portal engineered with semantic HTML5, CSS Grid, and dynamic booking simulation.
3. **MIVA Scholar Task Planner:** The interactive task management application built for this course with real-time analytics.
4. **AuthShield Security & Form Validator:** Client-side defensive utility calculating password entropy and auditing input integrity.

2.4 Academic Planner Page (`planner.html`)

A client-side task scheduling application built with vanilla JavaScript. Students can add academic milestones (title, enrolled course dropdown, target due date, priority), toggle completion with dynamic strike-through styling, delete tasks, and filter by status (All, Pending, Completed). All records are serialized into `localStorage` so tasks persist across browser sessions.

2.5 Contact Page (`contact.html`)

An accessible communication portal with Prince Daniel's email and phone number. The form strictly enforces three client-side JavaScript validation rules: (1) no field is left blank, (2) email conforms to RFC regular expression standards, and (3) phone numbers contain strictly numeric digits.

3. Assessment Rubric Compliance Matrix

Detailed mapping showing how every requirement from the official COS 106 term project brief (20 Marks) is satisfied:

Assessment Requirement	Implementation in Codebase	Source File(s)	Evidence & Status
1. Homepage Structure Student name, photo, welcome message, nav menu, biography, layout.	Hero section with profile card on the left side (`portrait.jpg`), matric details, sticky navbar, KPI boxes, and Royal Navy theme.	`index.html` `style.css`	**Verified** Lines 40–110
2. About Me Page Educational background, career roadmap, technical skills, hobbies, coursework.	Full degree details, 3-phase career roadmap, animated skill progress bars via `IntersectionObserver` , hobbies, and 8-course table.	`about.html` `js/main.js`	**Verified** Table lines 95–160
3. Projects Showcase At least 3 projects, descriptions, images, working or simulated links.	4 applied projects: ApexPay mobile wallet, CampusConnect event hub, MIVA Scholar planner, and AuthShield with category filters and interactive modal.	`projects.html` `js/projects.js`	**Verified** Cards lines 80–190
4. Academic Planner (Add Task) Form to schedule academic tasks with validation.	Dynamic form collecting title, course dropdown, due date, and priority. Unshifts into task array, saves to `localStorage` , and renders to DOM.	`planner.html` `js/planner.js`	**Verified** Section 4.2
5. Planner (Complete Task) Mark tasks completed with dynamic visual state.	Checkbox toggle switches `completed: true` , applies `line-through` styling, updates progress bar, and logs completion toast.	`planner.html` `js/planner.js`	**Verified** `style.css:485`
6. Planner (Delete Task) Delete individual or completed tasks.	Delete button triggers confirmation modal/prompt, filters task from array, serializes updated state to `localStorage` , and redraws DOM.	`planner.html` `js/planner.js`	**Verified** Section 4.2
7. Contact Form (Fields) Full Name, Email Address, Phone Number, Message.	Accessible HTML5 `<form>` with explicit `<label>` tags, descriptive placeholders, and error feedback `<div>` containers.	`contact.html`	**Verified** Lines 90–165
8. Contact (Strict Validation) No empty field; valid email regex; digits-only phone.	Real-time `input` , `blur` , and `submit` validation. Enforces non-empty values, standard RFC email regex, and strictly numeric phone (`/^[0-9]+$/`).	`contact.html` `js/contact.js`	**Verified** Section 4.1
9. HTML Requirements Semantic elements, forms, tables, images, hyperlinks, lists, multimedia.	Full semantic structure (`<header>` , `<nav>` , `<main>` , `<section>` , `<table>`), functional `<audio controls>` , and responsive `<video controls>` .	All 5 HTML pages	**Verified** Section 4.3
10. CSS Requirements External CSS, responsive layout, Flexbox/Grid, transitions, mobile nav.	Central stylesheet (`style.css`), custom Royal Navy tokens, multi-column Flexbox/Grid layouts, mobile bottom app bar, and smooth micro-animations.	`style.css`	**Verified** Section 4.4
11. JavaScript Requirements Event handling, DOM updates, form validation, dynamic arrays.	Modular ES6+ scripts handling click/input/submit events, dynamic task rendering, modal dialogues, category filtering, and `localStorage` persistence.	`js/main.js` `js/planner.js` `js/contact.js`	**Verified** Sections 4.1 & 4.2

4. Key Source Code Implementations

4.1 Client-Side Validation Logic

File: js/contact.js

```
// Strict Email Validation using RFC Standard Regex
if (input.id === 'contactEmail') {
  if (!val) {
    setFieldError(input, errorEl, 'Email address is required.');
    return false;
  }
  const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
  if (!emailRegex.test(val)) {
    setFieldError(input, 'Please provide a valid email format (e.g., student@miva.edu.ng).');
    return false;
  }
  clearFieldError(input, errorEl);
  return true;
}

// Phone Number Validation: Strictly Digits Only (/^[0-9]+$/)
if (input.id === 'contactPhone') {
  if (!val) {
    setFieldError(input, errorEl, 'Phone number is required.');
    return false;
  }
  const digitsRegex = /^[0-9]+$/;
  if (!digitsRegex.test(val)) {
    setFieldError(input, errorEl, 'Phone number must contain digits only (no letters or symbols).');
    return false;
  }
  if (val.length < 8 || val.length > 15) {
    setFieldError(input, errorEl, 'Phone number length must be between 8 and 15 digits.');
    return false;
  }
  clearFieldError(input, errorEl);
  return true;
}
```

4.2 Interactive Task Management Engine

File: js/planner.js

```
// Add Milestone: Creates task object, unshifts into array, updates DOM and storage
function handleAddTask(e) {
  e.preventDefault();
  const newTask = {
    id: 'task-' + Date.now(),
    title: title,
    course: course,
    dueDate: dueDate,
    priority: priority,
    completed: false,
    createdAt: Date.now()
  };
  tasks.unshift(newTask);
  saveTasks();    // Serializes array into localStorage: JSON.stringify(tasks)
  renderTasks();  // Redraws DOM list and recalculates completion metrics
  e.target.reset();
}

// Complete Task: Toggles completed flag and updates progress track
function toggleTaskComplete(taskId) {
  const task = tasks.find(t => t.id === taskId);
  if (task) {
    task.completed = !task.completed;
    saveTasks();
    renderTasks();
  }
}

// Delete Task: Prompts confirmation, filters array, and updates UI
function deleteTask(taskId) {
  if (confirm('Are you sure you want to delete this academic task?')) {
    tasks = tasks.filter(t => t.id !== taskId);
    saveTasks();
    renderTasks();
  }
}
```

4.3 Semantic HTML Structure & Coursework Table

File: about.html

```
<!-- Semantic Coursework Summary Table with Scoped Column Headers -->
<table class="academic-table" summary="Table of Prince Daniel's registered 100-Level courses">
  <thead>
    <tr>
      <th scope="col">Course Code</th>
      <th scope="col">Course Title</th>
      <th scope="col">Credit Units</th>
      <th scope="col">Classification</th>
      <th scope="col">Target Grade</th>
    </tr>
  </thead>
  <tbody>
    <tr><td><strong>COS 106 / CSC 106</strong></td><td>Introduction to Web Technologies</td><td>3 Units</td><t
    <tr><td><strong>COS 102</strong></td><td>Problem Solving & Algorithmic Logic</td><td>3 Units</td><td>Core
  </tbody>
</table>

<!-- Native HTML5 Multimedia Integration -->
<audio controls preload="metadata"><source src="assets/media/intro-audio.wav" type="audio/wav"></audio>
<video controls poster="portrait.jpg" preload="metadata"><source src="assets/media/demo-video.mp4" type="vide
```

4.4 Royal Navy CSS Architecture & Mobile Navigation

File: style.css

```css
/* Royal Navy Blue Design Tokens & Strike-Through Completed Task Styling */
:root {
  --navy-dark: #0A192F;
  --navy-primary: #1E3A8A;
  --blue-electric: #38BDF8;
}

.task-item.completed {
  opacity: 0.65;
  background: var(--bg-card-subtle);
}
.task-item.completed .task-title {
  text-decoration: line-through;
  color: var(--text-subtle);
}

/* Floating Mobile App Bottom Bar */
@media (max-width: 768px) {
  .mobile-bottom-nav {
    display: flex;
    position: fixed;
    bottom: 0; left: 0; right: 0;
    height: 60px;
    background: var(--navy-dark);
    z-index: 9999;
  }
}
```

5. Cross-Device Testing & Verification

Testing was performed across multiple viewports and rendering engines to verify complete cross-browser compliance:

Target Test Feature	Testing Method & Environment	Observed Result & Verification
Multi-Page Navigation	Google Chrome (Desktop, 1920 × 1080) *Physical Testing Environment*	All links between Home, About, Projects, Planner, and Contact load with HTTP 200 OK. Active page indicators highlight accurately in the navigation bar.
Form Input Validation	Microsoft Edge (Chromium Engine) *Physical Testing Environment*	Empty submissions trigger red borders and error text. Invalid email format is rejected. Phone field strictly rejects alphabetical and special characters.
Task CRUD & Persistence	Google Chrome DevTools Mode *Emulated iPhone 14 (390 × 844)*	Milestones add smoothly, checkbox triggers strike-through line, delete removes item, and data reloads intact after hard browser refresh via `localStorage`.
Mobile Layout & Bottom Bar	Chrome Mobile (Android Viewport) *Mobile Responsive Verification*	Sticky bottom app bar provides instant thumb navigation; grid collapses to single column; touch targets measure ≥ 48px; zero horizontal scrolling.

6. Student Reflection & Cybersecurity Takeaways

Building this platform provided valuable insights into the direct intersection between frontend engineering and cybersecurity:

1. **Client-Side Validation vs. Defense-in-Depth:** While JavaScript form validation provides instant feedback for user experience, client-side controls can easily be bypassed by intercepting HTTP payloads or disabling scripts. True defense-in-depth requires robust server-side sanitization and parameterized queries.
2. **Document Structure & Predictability:** Implementing semantic HTML5 tags (`<header>`, `<nav>`, `<main>`, `<section>`, `<table>`) creates a deterministic document tree that enhances accessibility for screen readers and optimizes browser parsing.
3. **Zero-Dependency Architecture:** Developing the site entirely with pure Vanilla JavaScript and native CSS Flexbox/Grid eliminated third-party library dependencies, drastically reducing the software supply chain attack surface and ensuring lightning-fast load speeds.

7. Statement of Academic Originality

Declaration of Original Student Work

I, **PRINCE DANIEL**, confirm that this practical term project and the accompanying source code are my own original work, completed individually in accordance with the academic regulations of **MIVA Open University** for **COS 106: Introduction to Web Technologies** (20 Marks).

Prince Daniel
Matriculation No: 2025/C/CYB/1333 • Student ID: 301897306

Date: November 2026
Department of Cybersecurity • MIVA Open University